



**UNIVERSIDAD
DON BOSCO**

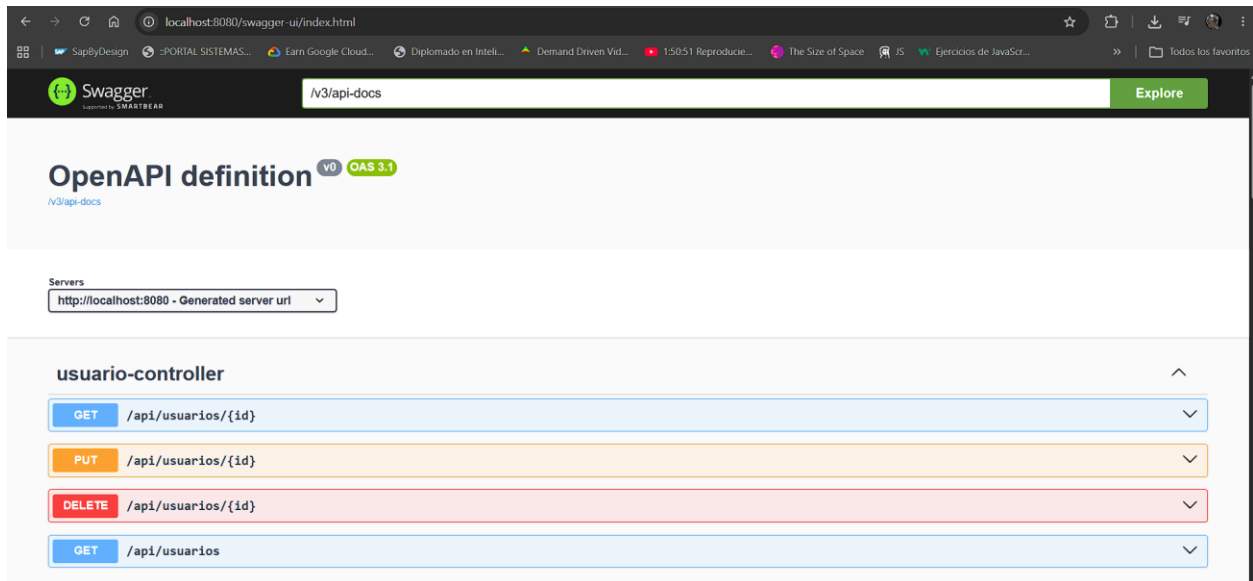
Alejandro Marcelo Aleman Ayala --AA240390

Odir Ezequiel Diaz Penate --DP240089

Proyecto de CATEDRA-DWF-TEORIA

Endpoints de nuestra API

Para nuestra API usamos diferentes endpoints los cuales los hemos registrado tanto como en swagger y de manera convencional en thunderclient para ir manejando las pruebas a ver si funciona correctamente todo



- **/api/usuarios**

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/usuarios/1' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3VzZXQ8bnVGL7h2v-h1PjFuqZkXg1wp8pncVEBdfR8'
```

Request URL

http://localhost:8080/api/usuarios/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "nombre": "Administrador SERMON", "correo": "admin@sermon.com", "rol": "ADMIN", "activo": true }</pre> Response headers <pre>cache-control: no-cache,no-store,max-age=0,must-revalidate connection: keep-alive content-type: application/json date: Mon, 18 May 2026 00:02:17 GMT expires: 0 keep-alive: timeout=60 pragma: no-cache transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers x-content-type-options: nosniff x-frame-options: DENY x-xss-protection: 0</pre>

Responses

Code	Description	Links
------	-------------	-------

En nuestro primer Endpoint nos devuelve los usuarios creados que solo seria técnico y administrador dependiendo el ID que utilicemos, como por ejemplo en la captura el id 1 pertenece al administrador

- **/api/usuarios (PUT)**

usuario-controller

GET /api/usuarios/{id}

PUT /api/usuarios/{id}

Parameters

Name	Description
id * required	integer(\$int64) (path)

Request body required

application/json

```
{
  "id": 0,
  "nombre": "alejandro",
  "correo": "alejandro@sermon.com",
  "password": "123",
  "rol": "ADMIN",
  "activo": true
}
```

Este endpoint nos ayuda a registrar nuevos usuarios como se muestra en la captura y el resultado de éxito en la siguiente captura

Request URL

http://localhost:8080/api/usuarios/1

Server response

Code 200 Details

Response body

```
{
  "id": 1,
  "nombre": "alejandro",
  "correo": "alejandro@sermon.com",
  "rol": "ADMIN",
  "activo": true
}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Mon, 18 May 2026 00:12:43 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

- **Api/usuarios (GET)**

En este endpoint nos traemos los usuarios que están dentro del sistema ya sea administrador que para mejor uso solo hemos asignado 1 y el de técnico para que puedan ingresar a su debido mantenimiento después en frontend lo vamos a poder ver mejor

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/tecnicos/1' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1bzIzLCJ0eXB2IjI0IjH2G1pbm51ZXZvQW1kbW15bj28IjCjpb2w01jBRE1jT1I1ImlhdCI6MTkzOTAzMjkzNSwiZXN1IjoxNjc3SMTUwMzki1Q.2wPe1YAYGxNkAtK72kzk5IwYadmYrefbYtEnE6KJNhFOV0dM6
```

Request URL

```
http://localhost:8080/api/tecnicos/1
```

Server response

Code

200

Details

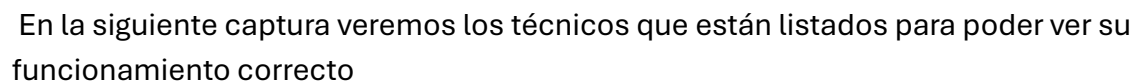
Response body

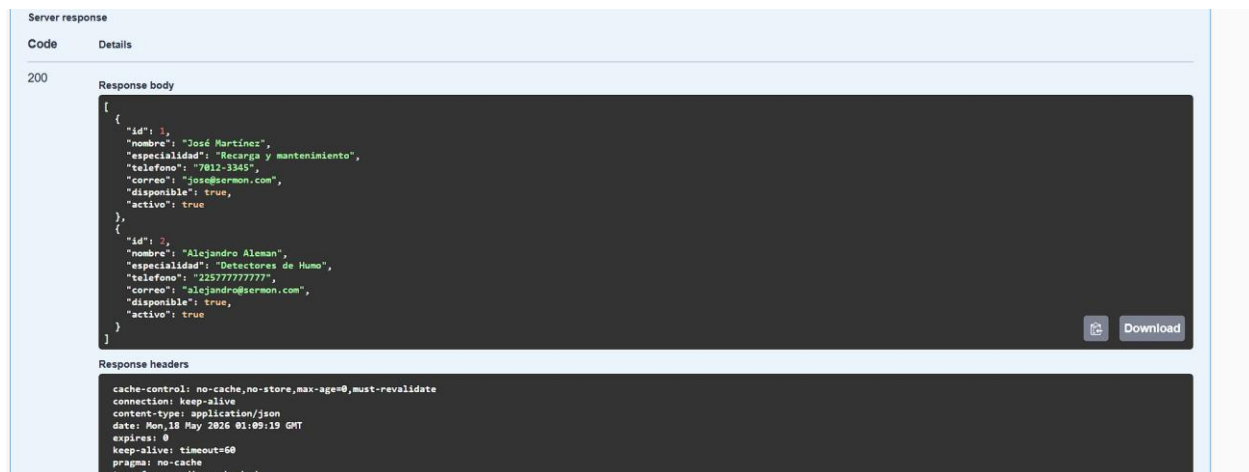
```
{
  "id": 1,
  "nombre": "Jos\u00e9 Mart\u00ednez",
  "especialidad": "Recarga y mantenimiento",
  "telefono": "7012-3345",
  "correo": "jose@sermon.com",
  "disponible": true,
  "activo": true
}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Mon, 18 May 2026 00:29:50 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

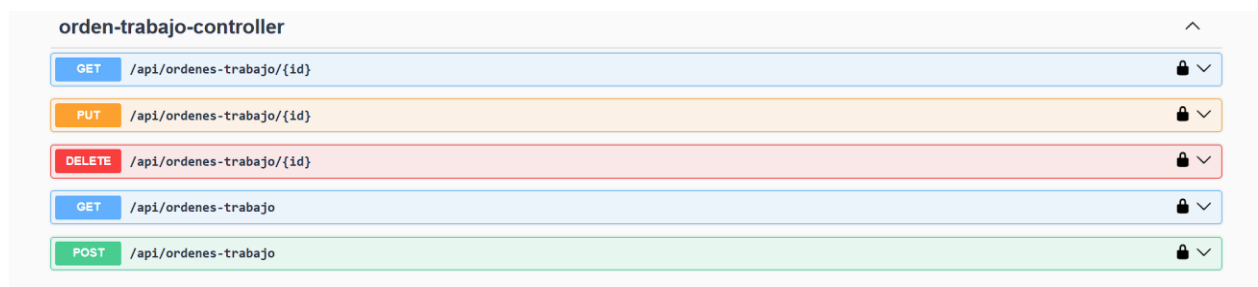
Dentro de este endpoint esperamos como respuesta todos los técnicos que están listados dentro de nuestra aplicación



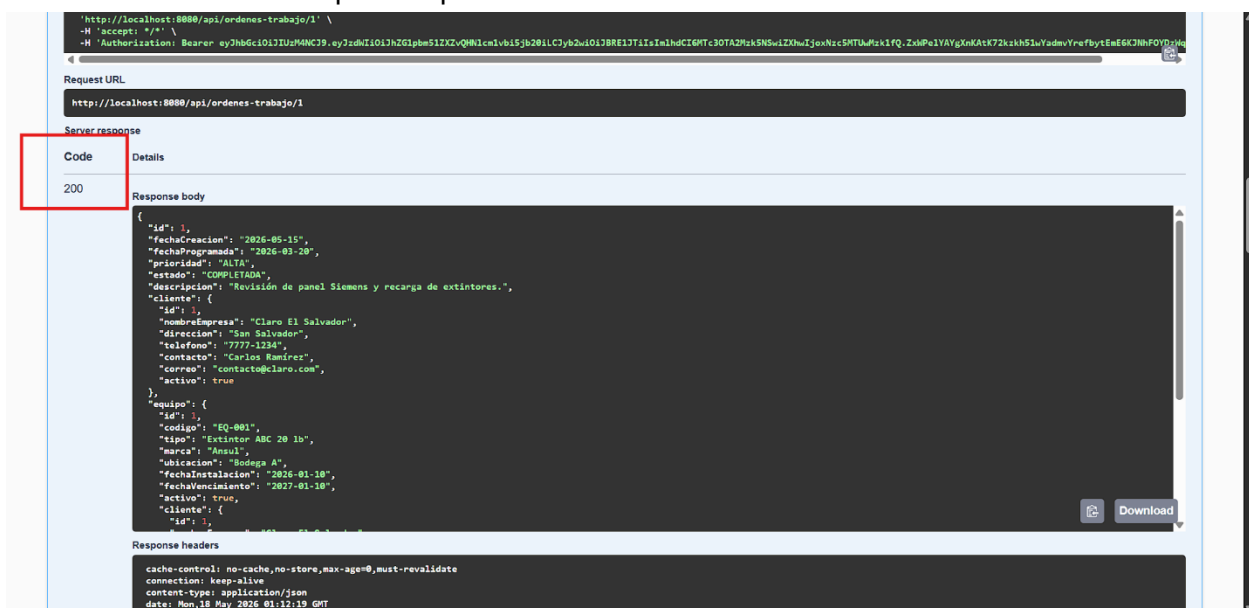


- **Api/ordenes-trabajo/**

Dentro de este endpoint vamos a esperar diferente tipo de información como puede ser los mantenimientos que hemos realizado o los que si necesitan en si un reporte que seria los que el usuario desea realizar



En la siguiente captura vamos a ver diferentes reportes que hemos hecho, tanto como simulando como de prueba para su correcto funcionamiento



- **Api/ordenes-trabajo (PUT)**

En la siguiente captura veremos como se puede agregar un reporte desde mantenimiento por eso toma diferentes ID y tiene una relación directa con diferentes endpoints también para llevar una mejor lógica y concordancia dentro de la aplicación

PUT /api/ordenes-trabajo/{id}

Parameters

Try it out

Name	Description
id * required	id
integer(\$int64)	(path)

Request body required

application/json

Example Value | Schema

```
{
  "id": 0,
  "fechaCreacion": "2026-05-18",
  "fechaProgramada": "2026-05-18",
  "prioridad": "string",
  "estado": "string",
  "descripcion": "string",
  "cliente": {
    "id": 0,
    "nombreEmpresa": "string",
    "direccion": "string",
    "telefono": "string",
    "contacto": "string",
    "correo": "string",
    "activo": true
  },
  "equipo": {
    "id": 0,
    "codigo": "string",
    "tiempo": "string"
  }
}
```

GET /api/ordenes-trabajo

Parameters

Cancel

No parameters

Execute

Responses

Code	Description	Links
200	OK	No links

Media type

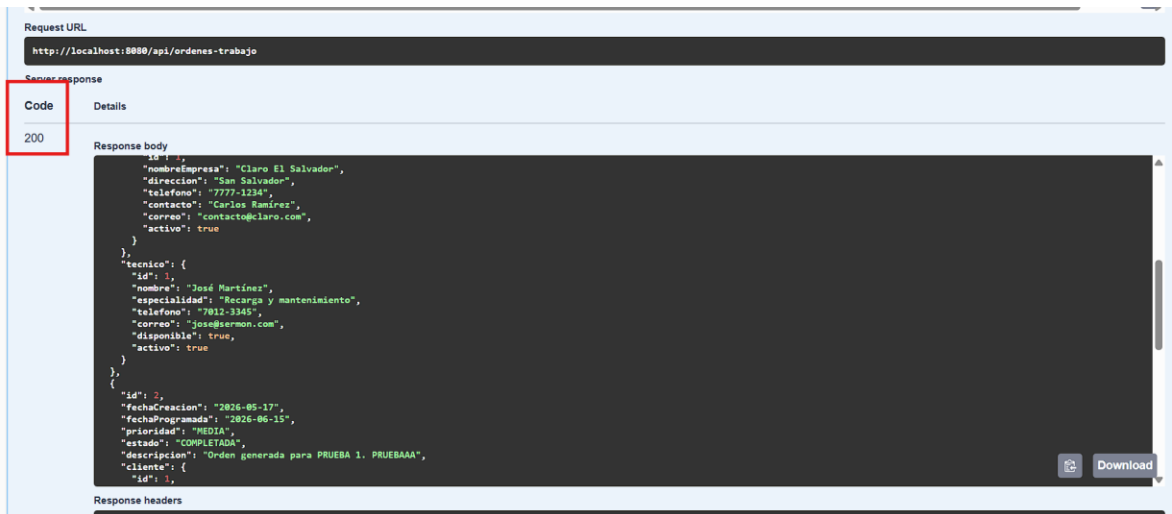
/

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "fechaCreacion": "2026-05-18",
  "fechaProgramada": "2026-05-18",
  "prioridad": "string",
  "estado": "string",
  "descripcion": "string",
  "cliente": {
    "id": 0,
    "nombreEmpresa": "string",
    "direccion": "string",
    "telefono": "string",
    "contacto": "string",
    "correo": "string",
    "activo": true
  },
  "equipo": {
    "id": 0,
    "codigo": "string",
    "tiempo": "string"
  }
}
```

En la siguiente captura vamos a ver las diferentes ordenes de trabajo que están declaradas en el frontend



- **api/mantenimiento-controller/**

Dentro de este apartado de endpoints encontraremos los mantenimientos que se han realizado, listar mantenimiento o poder borrarlo

mantenimiento-controller		^
GET	/api/mantenimientos/{id}	🔒
PUT	/api/mantenimientos/{id}	🔒
DELETE	/api/mantenimientos/{id}	🔒
GET	/api/mantenimientos	🔒
POST	/api/mantenimientos	🔒

Primero vamos a probar el endpoint numero 1 el cual nos mostrara un mantenimiento según si ID nos lo traerá

GET /api/mantenimientos/{id} 🔒

Parameters

Cancel

Name	Description
id * required integer(\$int64) (path)	1

Execute

Responses

Code	Description	Links
200	OK	No links

Media type

- **api/equipo-controller/**

equipo-controller				
GET	/api/equipos/{id}		🔒	⌵
PUT	/api/equipos/{id}		🔒	⌵
DELETE	/api/equipos/{id}		🔒	⌵
GET	/api/equipos		🔒	⌵
POST	/api/equipos		🔒	⌵

En este endpoint nos ayudare a listar los diferentes equipos que pueden estar disponibles para cada mantenimiento

GET /api/equipos/{id}

Parameters

Name	Description
id * required	
integer(\$int64)	2
(path)	

Execute

Clear

Dentro de ese endpoint nos traerá el equipo y con quien esta ligado ese equipo o asignado para cada mantenimiento

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/equipos/2' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZXQvQW1lc1VhIjB2bW1C3yb2w1O1BRE1J1IiwiaWF0IjE1InIhZCI6MTc3OTAzMjk5NSw1ZXh1IjoxNzUwNzki1FQ-ZsMPE1YAYgXnKAtK72kxh51uYadmVYrefBytEm6KJmF0YDQm
```

Request URL

http://localhost:8080/api/equipos/2

Server response

Code

200

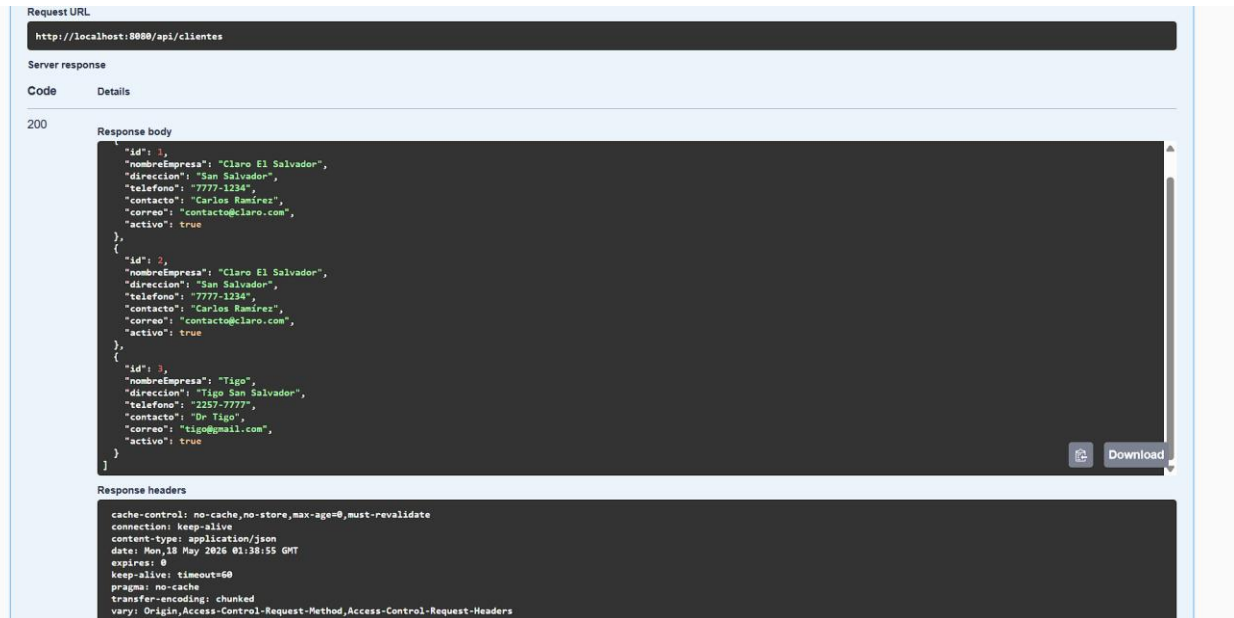
Details

Response body

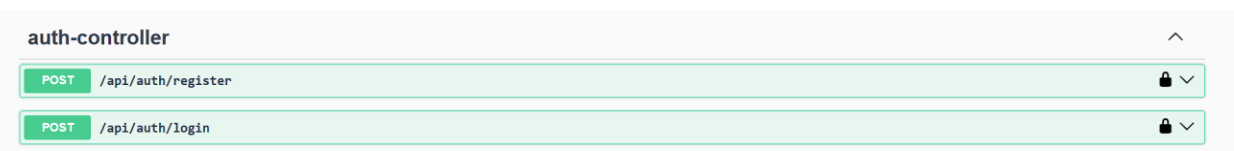
```
{
  "id": 2,
  "codigo": "2",
  "tipo": "Detector de Humo SIMPLEX",
  "marca": "SIMPLEX",
  "ubicacion": "San Salvador",
  "fechaInstalacion": "2026-05-27",
  "fechaVencimiento": "2026-05-28",
  "activo": true,
  "cliente": {
    "id": 3,
    "nombreEmpresa": "Tigo",
    "direccion": "Tigo San Salvador",
    "telefono": "2252-7777",
    "contacto": "Dr Tigo",
    "correo": "tigo@mail.com",
    "activo": true
  }
}
```

Response headers

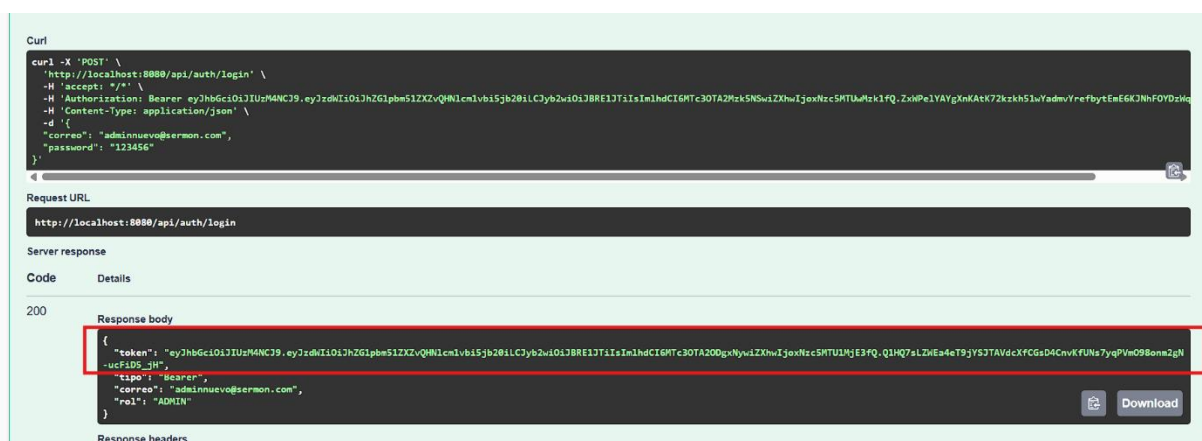
Como vamos a poder ver en la siguiente captura dentro del endpoint ed GET nos listara los diferentes clientes que están trabajando con nosotros



- **api/auth/controller**



En estos endpoint son de hacer POST donde vamos a recibir los token que nos ayudaran a iniciar sesión para entrar en la pagina web y poder ocupar los diferentes endopints ya que están protegidos con baerer como lo solicita el desafio



Dentro de los últimos EndPoint han sido pensados para que el frontend se vea de una mejor manera con un pequeño resumen de diferetes mantenimientos con alertas interactivas y un pequeño dashboard para darle una mejor facilidad de manejo al administrador y al tecnico

reporte-controller

GET /api/reportes/resumen

dashboard-controller

GET /api/dashboard

alerta-controller

GET /api/alertas

Curl

curl -X 'GET' \n'http://localhost:8080/api/reportes/resumen' \n-H 'accept: */*' \n-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3N1bWU6IjE6ImNpdC9.eyJzdWIiOiJhZG1pbm51ZDZvQW1lcmlvbi5jb28iLCJyb2wiOiJBRRE1TjIiIn1hdCI6MTc3OTA2Mzk5NSwiZXNpdjoxNzc5SHTUwMzkiFQ.ZxMPE1YAYgXnKAtK72kzh51uYadmYrefBytEmE6KJNHFOYDzhM'

Request URL

http://localhost:8080/api/reportes/resumen

Server response

Code

Details

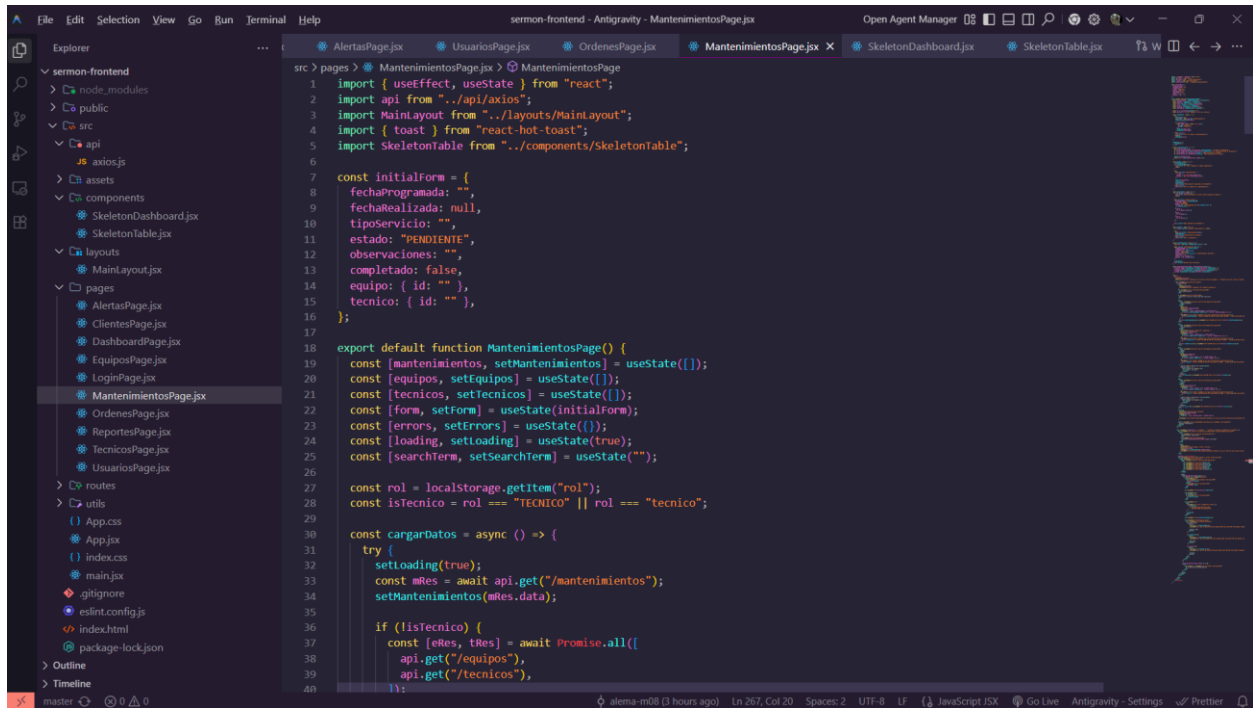
200

Response body

```
{
  "totalClientes": 3,
  "totalEquipos": 2,
  "totalTecnicos": 2,
  "totalMantenimientos": 4,
  "mantenimientosCompletados": 3,
  "mantenimientosPendientes": 1,
  "totalOrdenesTrabajo": 2,
  "ordenesPendientes": 0,
  "ordenesCompletadas": 2
}
```

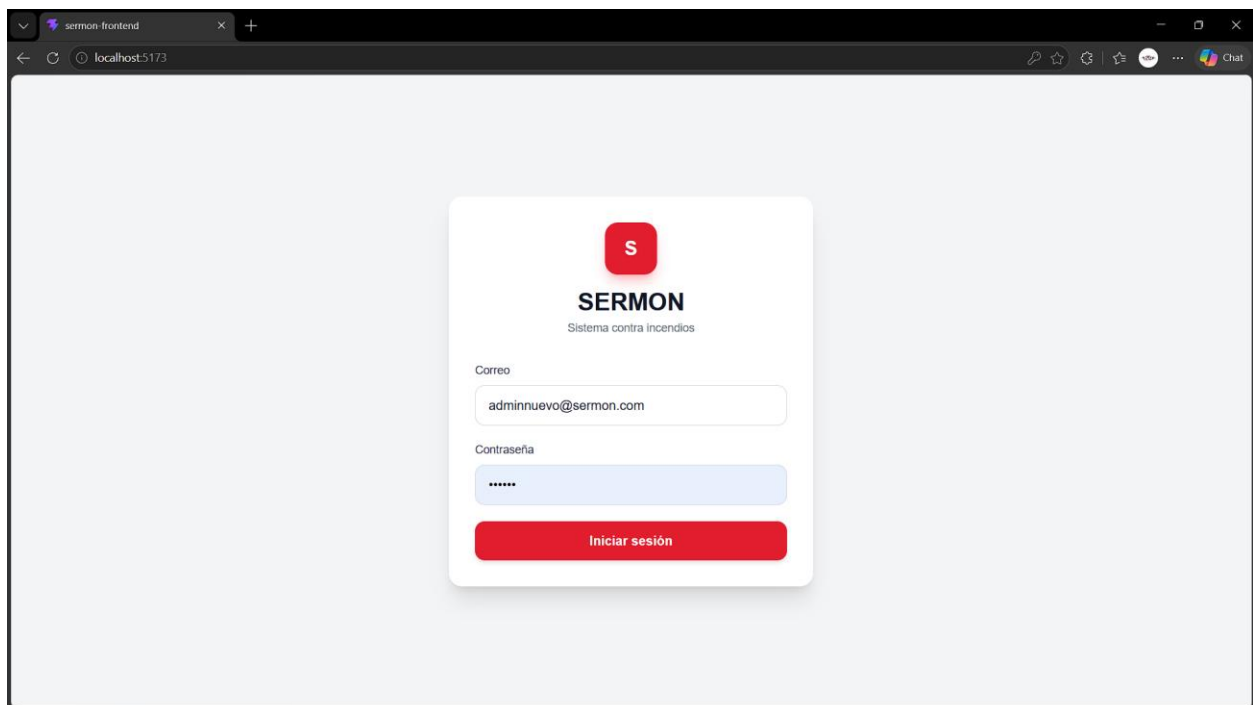
Download

FRONTEND CAPTURAS



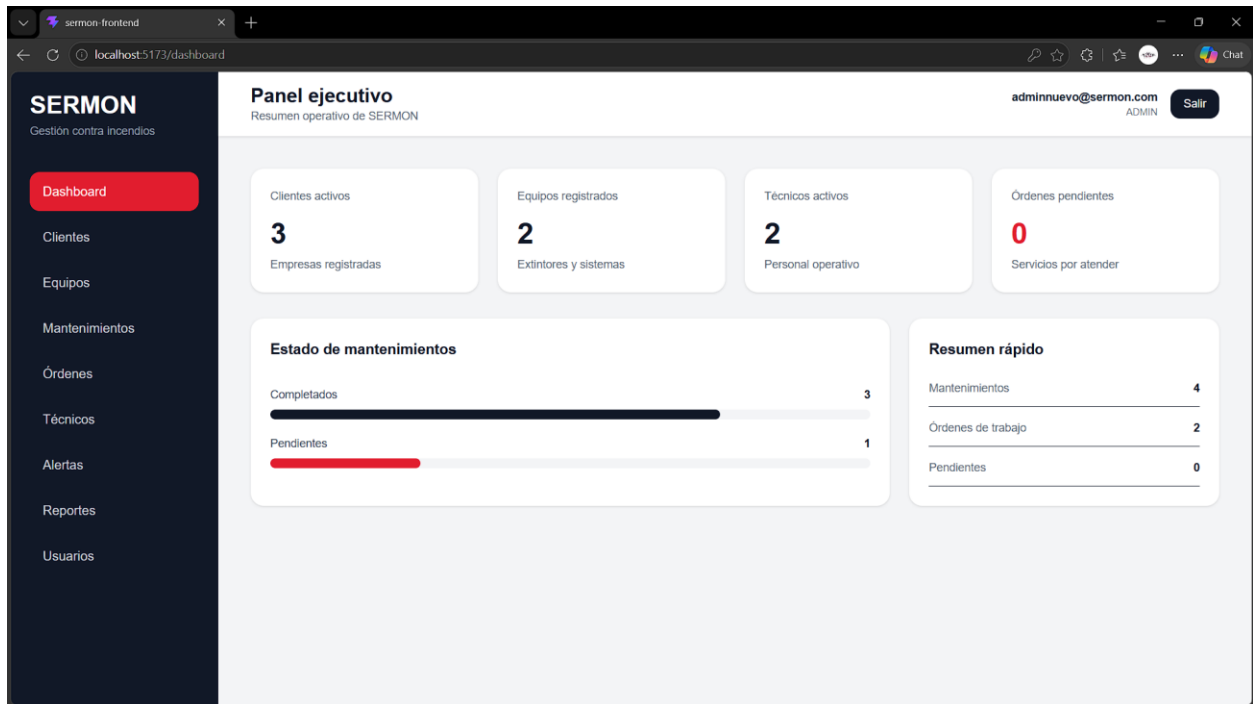
```
src > pages > MantenimientosPage.jsx > MantenimientosPage
1 import { useEffect, useState } from "react";
2 import api from "../api/axios";
3 import MainLayout from "../layouts/MainLayout";
4 import { toast } from "react-hot-toast";
5 import SkeletonTable from "../components/SkeletonTable";
6
7 const initialForm = {
8   fechaProgramada: "",
9   fechaRealizada: null,
10  tipoServicio: "",
11  estado: "PENDIENTE",
12  observaciones: "",
13  completado: false,
14  equipo: { id: "" },
15  tecnico: { id: "" },
16 };
17
18 export default function MantenimientosPage() {
19   const [mantenimientos, setMantenimientos] = useState([]);
20   const [equipos, setEquipos] = useState([]);
21   const [tecnicos, setTecnicos] = useState([]);
22   const [form, setForm] = useState(initialForm);
23   const [errors, setErrors] = useState({});
24   const [loading, setLoading] = useState(true);
25   const [searchTerm, setSearchTerm] = useState("");
26
27   const rol = localStorage.getItem("rol");
28   const isTecnico = rol === "TECNICO" || rol === "tecnico";
29
30   const cargarDatos = async () => {
31     try {
32       setLoading(true);
33       const mRes = await api.get("/mantenimientos");
34       setMantenimientos(mRes.data);
35
36       if (!isTecnico) {
37         const [eRes, tRes] = await Promise.all([
38           api.get("/equipos"),
39           api.get("/tecnicos"),
40         ]);
41       }
42     } catch (error) {
43       console.log(error);
44     }
45   };
46
47   useEffect(() => {
48     cargarDatos();
49   }, []);
50
51   return (
52     <MainLayout>
53       <div>
54         <h2>Mantenimientos</h2>
55         <div>
56           <input type="text" value={searchTerm} />
57           <button>Buscar</button>
58         </div>
59         <table>
60           <thead>
61             <tr>
62               <th>ID</th>
63               <th>Fecha Programada</th>
64               <th>Fecha Realizada</th>
65               <th>Tipo Servicio</th>
66               <th>Estado</th>
67               <th>Observaciones</th>
68               <th>Equipo</th>
69               <th>Tecnico</th>
70             </tr>
71           </thead>
72           <tbody>
73             <tr>
74               <td>1</td>
75               <td>2023-10-26</td>
76               <td>2023-10-26</td>
77               <td>Revisión</td>
78               <td>PENDIENTE</td>
79               <td>Revisión general del sistema</td>
80               <td>Equipo 1</td>
81               <td>Tecnico 1</td>
82             </tr>
83             <tr>
84               <td>2</td>
85               <td>2023-10-27</td>
86               <td>2023-10-27</td>
87               <td>Revisión</td>
88               <td>PENDIENTE</td>
89               <td>Revisión general del sistema</td>
90               <td>Equipo 2</td>
91               <td>Tecnico 2</td>
92             </tr>
93             <tr>
94               <td>3</td>
95               <td>2023-10-28</td>
96               <td>2023-10-28</td>
97               <td>Revisión</td>
98               <td>PENDIENTE</td>
99               <td>Revisión general del sistema</td>
100              <td>Equipo 3</td>
101              <td>Tecnico 3</td>
102            </tr>
103          </tbody>
104        </table>
105      </div>
106    </MainLayout>
107  );
108}
```

LOGIN



Dashboard principal

Dentro del Dashboard vamos a encontrar un pequeño resumen de diferentes mantenimientos, los técnicos y los equipos registrados dándonos así un panel minimalista pero con toda la información necesaria para administrar la empresa



Clientes

En la vista de clientes vamos a poder agregar y ver listados los otros clientes que ya tenemos registrados con sus diferentes funciones como eliminar o editar un cliente

The screenshot shows the 'Gestión de clientes' page. The left sidebar is the same as the dashboard. The main content area is titled 'Gestión de clientes' and 'Administra empresas atendidas por SERMON'. It is divided into two sections: 'Nuevo cliente' and 'Clientes registrados'. The 'Nuevo cliente' section contains a form with fields for Empresa, Dirección, Teléfono, Contacto, and Correo, followed by a 'Guardar' button. The 'Clientes registrados' section shows a search bar and a table of registered clients.

Empresa	Contacto	Teléfono	Estado	Acciones	
Claro El Salvador contacto@claro.com	Carlos Ramirez	7777-1234	Activo	Editar	Eliminar
Claro El Salvador contacto@claro.com	Carlos Ramirez	7777-1234	Activo	Editar	Eliminar
Tigo tigo@gmail.com	Dr Tigo	2257-7777	Activo	Editar	Eliminar

Equipos

Así mismo como la vista anterior en equipos tenemos las mismas funciones cambiando un poco el formulario por fines profesionales

SERMON
Gestión contra incendios

Dashboard
Clientes
Equipos
Mantenimientos
Órdenes
Técnicos
Alertas
Reportes
Usuarios

Inventario de equipos
Administra extintores, paneles y sistemas contra incendios

adminnuevo@sermon.com
ADMIN **Salir**

Registrar equipo

Código

Tipo

Marca

Ubicación

Fecha instalación

Fecha vencimiento

Cliente

Equipos registrados
Mostrando 2 resultados

Buscar código, marca, cliente...

Código	Equipo	Cliente	Vencimiento	Acciones
EQ-001	Extintor ABC 20 lb Ansul - Bodega A	Claro El Salvador	2027-01-10	Editar Eliminar
2	Detector de Humo SIMPLEX SIMPLEX - San Salvador	Tigo	2026-05-28	Editar Eliminar

Mantenimientos

En el apartado de mantenimientos vamos a tener diferentes tipos de operaciones o de funciones cómo puede ser Eliminar un mantenimiento y realizar una orden de ese mantenimiento que lo veremos en la siguiente captura más adelante. De igual manera tenemos un apartado para saber el estado de ese mantenimiento una vez el técnico o el administrador de click en mantenimiento completado el estado cambiará a completado y se podrá ver reflejado en el dashhboard principal

SERMON
Gestión contra incendios

Dashboard
Clientes
Equipos
Mantenimientos
Órdenes
Técnicos
Alertas
Reportes
Usuarios

Nuevo mantenimiento
Registra el servicio técnico que debe realizarse.

Fecha programada

Tipo de servicio

Equipo

Técnico

Observaciones (opcional)

Guardar mantenimiento

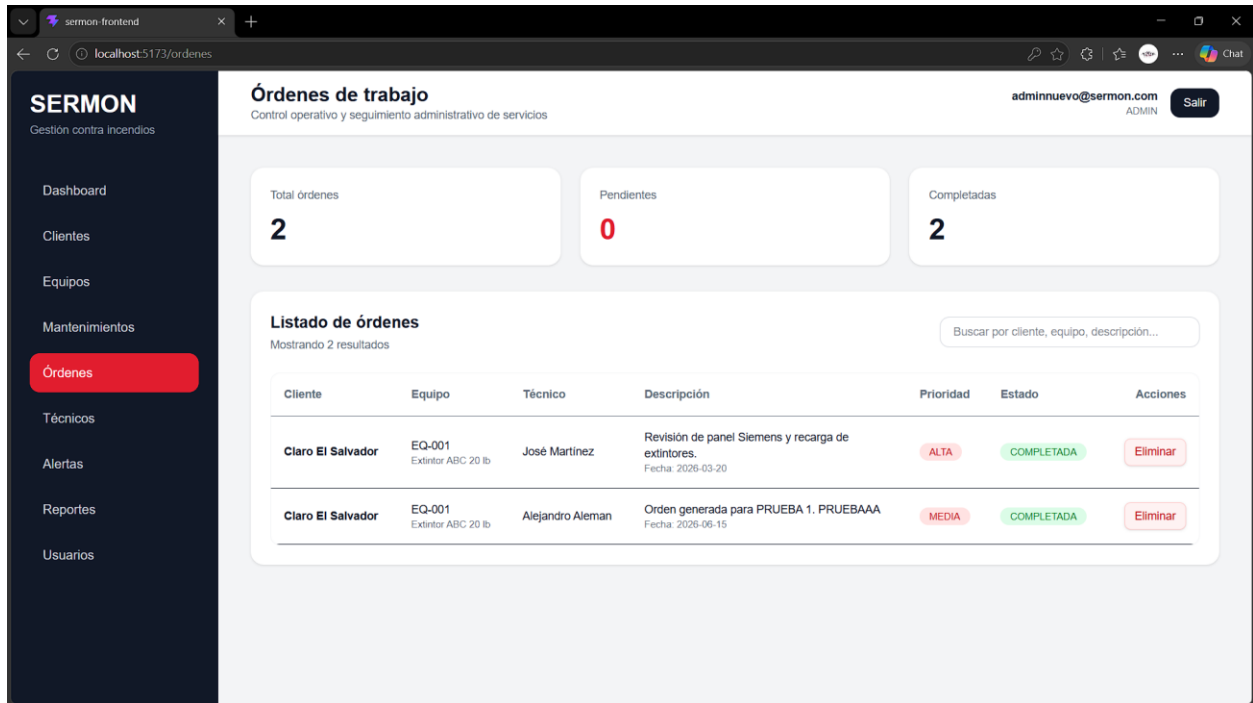
Mantenimientos registrados
Mostrando 4 resultados

Buscar equipo, técnico, servicio...

Técnico	Servicio	Estado	Fecha	Acciones
Alejandro Aleman	Detectores de humo Limpieza y mantenimiento de los detectores de humo	COMPLETADO	2026-05-19	Generar orden Eliminar
José Martínez	Prueba Prueba	COMPLETADO	2026-05-29	Generar orden Eliminar
Alejandro Aleman	PRUEBA 1 PRUEBAAA	PENDIENTE	2026-06-15	Generar orden Realizado Eliminar
Alejandro Aleman	PRUEBA PRUEBA PRUEBA PRUEBA	COMPLETADO	2026-05-21	Generar orden Eliminar

Órdenes

En el apartado de órdenes vamos a ver las siguientes funciones donde cabe mencionar la función de eliminar, en órdenes prácticamente es un pequeño resumen sobre los mantenimientos que se han dado es como el dashboard pero no siempre los mantenimientos van a tener órdenes ya que eso va a hacer realizados por el administrador si un mantenimiento requiere de una orden le dará click que se genere una orden y se verá reflejada en la vista de órdenes con sus diferentes estados



SERMON
Gestión contra incendios

Órdenes de trabajo
Control operativo y seguimiento administrativo de servicios

adminnuevo@sermon.com
ADMIN [Salir](#)

Total órdenes: **2**

Pendientes: **0**

Completadas: **2**

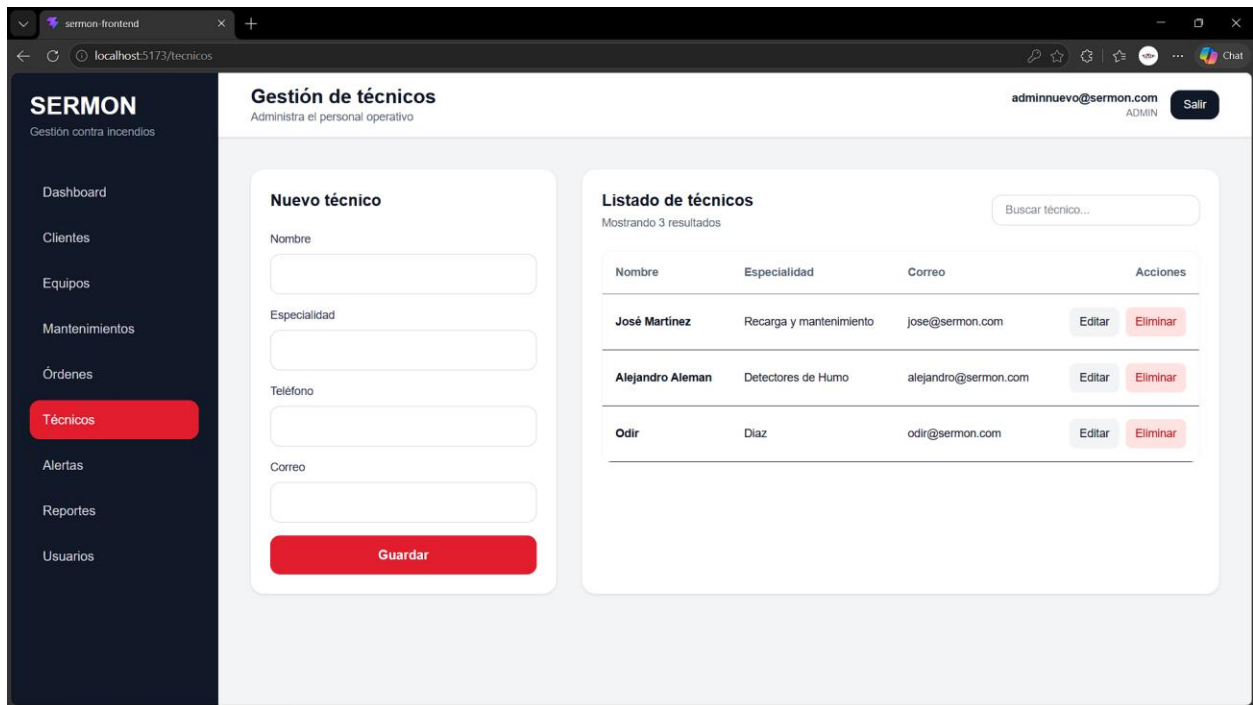
Listado de órdenes
Mostrando 2 resultados

Buscar por cliente, equipo, descripción...

Cliente	Equipo	Técnico	Descripción	Prioridad	Estado	Acciones
Claro El Salvador	EQ-001 Extintor ABC 20 lb	José Martínez	Revisión de panel Siemens y recarga de extintores. Fecha: 2026-03-20	ALTA	COMPLETADA	Eliminar
Claro El Salvador	EQ-001 Extintor ABC 20 lb	Alejandro Aleman	Orden generada para PRUEBA 1. PRUEBAAA Fecha: 2026-06-15	MEDIA	COMPLETADA	Eliminar

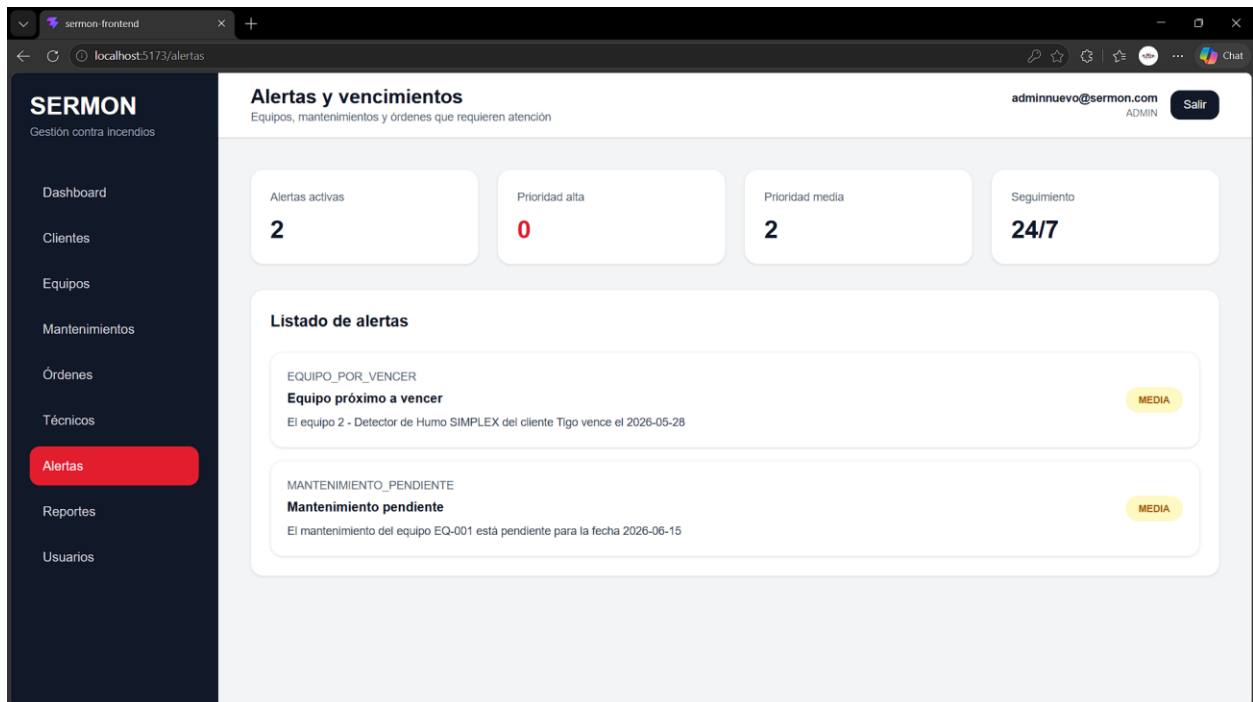
Técnicos

Dentro de la lista de técnicos vamos a encontrar un formulario que nos ayudará a agregar un nuevo técnico lo vamos a ver reflejado en la parte derecha de la vista como un pequeño resumen de los diferentes técnicos que tenemos con nosotros o en la empresa las acciones que podemos realizar como administrador con los técnicos sería eliminar y editar un técnico en cualquier caso cambie de teléfono u otra circunstancia



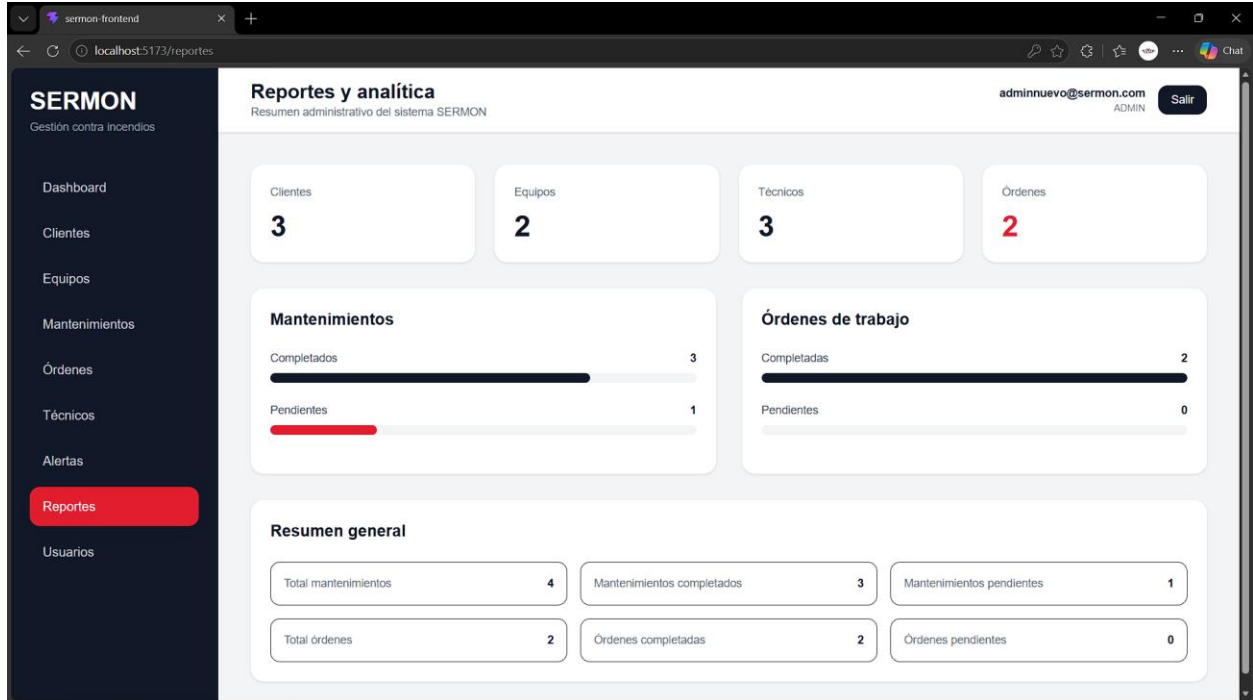
Alertas

Dentro de alertas encontraremos los equipos próximos a vencer o los distintos mantenimientos que tenemos pendientes



Reportes

Dentro de la vista de reportes vamos a encontrar un resumen mas detallado sobre los mantenimientos que hemos llevado acabo, los completados y los pendientes dando así una mejor visualización sobre los mantenimientos que aun no se han realizado



Usuarios

Gestión de usuarios
Administra cuentas y roles

adminnuevo@sermon.com ADMIN [Salir](#)

Usuarios del sistema
Mostrando 3 resultados

Buscar por nombre, correo o rol...

Nombre	Correo	Rol	Estado	Acciones
alejandro	alejandro@sermon.com	ADMIN	Activo	Editar Eliminar
Técnico SERMONN	tecnico@sermon.com	TECNICO	Activo	Editar Eliminar
Admin Nuevo	adminnuevo@sermon.com	ADMIN	Activo	Editar Eliminar

CAPTURAS TEST

The screenshot shows an IDE with the `AuthServiceImplTest` class open. The class contains the following code:

```
13 import org.mockito.InjectMocks;
14 import org.mockito.Mock;
15 import org.mockito.junit.jupiter.MockitoExtension;
16 import org.springframework.security.authentication.AuthenticationManager;
17 import org.springframework.security.crypto.password.PasswordEncoder;
18
19 import java.util.Optional;
20
21 import static org.junit.jupiter.api.Assertions.*;
22 import static org.mockito.Mockito.*;
23
24 @ExtendWith(MockitoExtension.class)
25 class AuthServiceImplTest {
26
27     @Mock
28     private UsuarioRepository usuarioRepository;
29
30     @Mock
31     private PasswordEncoder passwordEncoder;
```

The Run window shows the test results for `AuthServiceImplTest` (org.example.sermon.service). The tests passed are:

- ✓ registrar_debeCrearUsuarioYRetornarToken() 2 sec 547 ms
- ✓ login_debeRetornarTokenCuandoCredencialesSonCorrectas() 27 ms

The total test time is 2 sec 574 ms. The process finished with exit code 0.

The screenshot shows an IDE with the `ClienteServiceImplTest` class open. The class contains the following code:

```
19 @ExtendWith(MockitoExtension.class)
20 class ClienteServiceImplTest {
21
22     @Mock
23     private ClienteRepository clienteRepository;
24
25     @InjectMocks
26     private ClienteServiceImpl clienteService;
27
28     @Test
29     void listarTodos_debeRetornarListaClientes() {
30         Cliente cliente = Cliente.builder()
31             .id(1L)
32             .nombreEmpresa("Claro El Salvador")
33             .direccion("San Salvador")
34             .telefono("7777-1234")
35             .contacto("Carlos Ramirez")
36             .correo("contacto@claro.com")
37             .build();
```

The Run window shows the test results for `ClienteServiceImplTest` (org.example.sermon.service). The tests passed are:

- ✓ buscarPorId_cuandoNoExiste_debeLanzarExcepcion() 1 sec 849 ms
- ✓ eliminar_debeEliminarClienteExistente() 8 ms
- ✓ buscarPorId_cuandoExiste_debeRetornarCliente() 5 ms
- ✓ crear_debeGuardarClienteActivo() 13 ms
- ✓ listarTodos_debeRetornarListaClientes() 5 ms

The total test time is 1 sec 880 ms. The process finished with exit code 0.